

Tower Siege: Integrating C++ Data Structures and Python Algorithms in an Educational Tower-Defense Game

Daniel Santiago Pérez Madera
Facultad de Ingeniería
Universidad Distrital Francisco José de Caldas
Bogotá, Colombia
ID: 20231020203

Abstract—Educational software engineering projects demand that students reconcile algorithm theory with the real-time constraints of interactive applications, exposing the tension between asymptotic complexity and practical system performance. This work presents *Tower Siege*, a tower-defense game in which a C++ engine built without external libraries—with explicit raw-pointer memory management—maintains enemy waves in a hand-rolled singly linked list and tower rankings in an unbalanced Binary Search Tree, while a Python/Pygame front end computes Greedy tower-placement recommendations and Backtracking depth-first enemy paths, with all inter-process data exchanged exclusively through JSON files on disk. A fully playable five-level game was delivered and validated through eleven unit tests and end-to-end acceptance tests; however, a critical emergent interaction was identified in which the Greedy advisor’s castle-proximity weighting progressively narrows valid corridors until the Backtracking’s $O(4^{n,m})$ worst-case complexity causes perceptible game freezes, constituting the project’s primary open limitation and the target of future work.

Index Terms—tower defense, data structures, greedy algorithms, backtracking, pathfinding, raw pointers, C++, Pygame, game algorithms

I. INTRODUCTION

Project-based learning in introductory computer science courses forces students to bridge the gap between algorithm theory and the engineering demands of real-world software. Tower-defense games have emerged as a productive vehicle for this goal because they naturally interleave three classical problem domains: dynamic data-structure management for moving entities, heuristic search for placement advice, and shortest-path computation for adversarial routing [1]. The genre therefore allows a single semester project to simultaneously exercise data structures, algorithm design, software architecture, and interactive performance—constraints that rarely coexist in purely algorithmic exercises.

Pathfinding in game environments has been studied extensively since the introduction of the A^* algorithm by Hart, Nilsson, and Raphael [2], which combines a backward cost with an admissible heuristic to find shortest paths with substantially lower search effort than exhaustive methods. Backtracking depth-first search (DFS), while asymptotically inferior, remains pedagogically valuable because it makes search-

space growth and pruning tangible [1]. For tower-placement advice, greedy heuristics that score candidate cells according to incoming threat or path frequency are common in game-algorithm design because they run in linear time over the grid and produce immediate visual feedback [1]. Both strategies, however, interact non-trivially when they share a problem domain: placement decisions alter the search space of the pathfinder, and a biased placement heuristic can systematically worsen pathfinding performance.

Dynamic entity management in game engines typically relies on linked lists for $O(1)$ append and sequential traversal, and on search trees for priority-ordered access [1]. Implementing these structures from scratch—without standard library containers such as `std::list` or `std::map`—requires explicit raw-pointer management, a skill central to systems programming courses.

For inter-process communication between the C++ engine and the Python front end, JSON is a natural exchange format [3]. While mature C++ JSON libraries such as *nlohmann/json* [4] simplify parsing considerably, course constraints mandating zero external dependencies required a hand-written serialiser and parser using only the C++ standard library. On the Python side, Pygame [5] provides the rendering, event, and timing primitives needed to build a responsive game loop without additional dependencies.

This paper makes the following contributions. First, it describes a complete two-language game engine that couples a C++ data-structure layer to a Python algorithm and UI layer through a clean, file-based JSON contract. Second, it documents a systematic emergent interaction—the Greedy–Backtracking conflict—in which a locally optimal placement heuristic gradually degrades the performance of an exhaustive pathfinder to the point of causing game freezes, and proposes two concrete remediation strategies. The remainder of this paper is organised as follows: Section II describes system architecture, data-structure implementations, and algorithm design; Section III presents testing results and analyses the conflict; Section IV summarises achievements and outlines future work.

II. METHODS AND MATERIALS

A. System Architecture

Tower Siege is structured in three independent layers whose coupling is intentionally minimised to permit isolated testing and clear separation of concerns, as illustrated in Fig. 1. The *C++ engine* is a standalone binary responsible for all game-state mutations: it reads a command from `data/input.json`, applies it, and writes the resulting state to `data/state.json`. It has no dependency on Python, Pygame, or any external library. The *Python layer* hosts the Pygame user interface, the Greedy and Backtracking algorithm modules, and the bridge that serialises player actions into `input.json` and deserialises engine responses from `state.json`. The only runtime coupling between the two processes is the pair of JSON files on disk; there are no sockets, shared memory regions, or inter-process pipes carrying data. This strict decoupling was chosen because it allows each layer to be developed and unit-tested independently, and because it mirrors microservice architectural patterns in which a well-defined data contract is the sole interface between components.

B. C++ Engine and Data Structures

The design philosophy of the C++ engine reflects two constraints: correctness requirements enforced by the game loop, and a course mandate that all data structures be implemented from raw pointers without any STL container that abstracts away memory layout. Both constraints converged on the same idiomatic style: a C-style API with an explicit `init` function that zeroes out the structure and an explicit `free` function that walks the heap-allocated chain and releases every node before nullifying the head pointer. This pattern makes memory ownership unambiguous and eliminates hidden allocations that would complicate debugging.

The enemy wave is maintained as a singly linked list (`EnemyList` in `linked_list.h/cpp`). Each node (`EnemyNode`) holds a copy of the `Enemy` struct and a raw next pointer; the list handle exposes both head and tail pointers so that `ll_push_back` runs in $O(1)$ without traversal. Removal of a dead enemy (`ll_remove_id`) requires an $O(n)$ scan because the list is ordered by insertion time, not by ID; given the small wave sizes (at most ten enemies simultaneously), this is acceptable. Each engine step, `ll_advance` walks the list once and advances every enemy along its pre-computed path by `speed` cells, achieving $O(n)$ movement resolution.

Towers are stored in an unbalanced Binary Search Tree (`TowerTree` in `BSTree.h/tree.cpp`), keyed on `attack_power`. The primary purpose of the BST is to produce a sorted tower listing on every engine call via `tt_inorder`, which performs an in-order traversal and appends towers to a `std::vector` in ascending power order; this vector is serialised into `state.json` and rendered in the HUD panel (visible in Fig. 3), directly demonstrating the BST property at runtime. Insertion (`tt_insert`) runs in $O(h)$

where h is the tree height; because the two distinct attack-power values used in the project (15 and 40) bound h to 2, all operations are effectively constant time in practice, though the implementation is general and handles arbitrary keys.

The engine's JSON handling is fully hand-written in `JsonIO.cpp`: it implements an iterative parser for objects, arrays, strings, and numbers, and a corresponding serialiser. No external library is used. The four supported commands—"init", "step", "place_tower", and "remove_tower"—cover the complete lifecycle of the game state; for the non-step commands the engine rebuilds its data structures and echoes the state back without modification, allowing Python to verify round-trip correctness.

C. Python Algorithm Layer

The Greedy advisor (`game/algorithms/greedy.py`) is a pure function that receives the grid dimensions, castle position, current tower list, and enemy list, and returns the free cell with the highest scalar threat score. The score for a candidate cell (r, c) is defined as

$$S(r, c) = \sum_{\substack{e \in \text{enemies}, \\ d_e \leq 7}} (8 - d_e) \frac{e_{\text{hp}}}{100} + \max(0, 8 - d_{\text{castle}}) \cdot 1.5 \quad (1)$$

where $d_e = |r - e_r| + |c - e_c|$ is the Manhattan distance from the candidate cell to enemy e , and d_{castle} is the Manhattan distance to the castle cell. The first term rewards cells that are close to multiple high-health enemies; the second term applies a uniform proximity bonus to cells near the castle. The function runs in $O(nm \cdot |\text{enemies}|)$ time, which on a 10×10 grid with ten enemies takes under a millisecond and adds no perceptible latency to the game loop.

The Backtracking pathfinder (`game/algorithms/backtracking.py`) implements a DFS from a spawn point to the castle, avoiding tower-occupied cells. The algorithm maintains a `visited` array to prevent revisiting cells, a mutable `path` stack, and a `best` variable holding the shortest path found so far. A branch is pruned when a solution is already known and the partial path length plus the Manhattan lower bound to the goal is not strictly less than `len(best)`. Without this optimisation the DFS explores every route; with it, later branches are cut as soon as the first solution is found and the bound tightens. Worst-case complexity remains $O(4^{nm})$ because the first path may be found only after exhaustive exploration—a property exploited by the bug documented in Section III-C. The Backtracking is also used as a placement guard: `placing_blocks_path` in `main.py` tentatively adds the requested tower to the layout and invokes `find_path` for every spawn point; if any returns an empty list, the placement is silently rejected before the engine is even called, preventing the player from sealing all enemy routes.

D. Pygame User Interface and Level Design

The Pygame front end runs at 30 frames per second with a one-second engine-step interval and a per-level enemy spawn

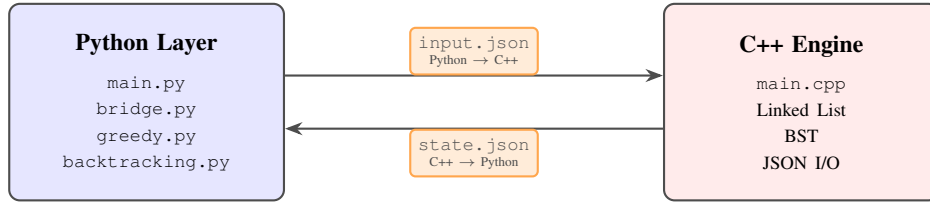


Fig. 1. System architecture. The Python layer writes `data/input.json` (player command + pre-computed paths) and reads `data/state.json` (updated game state). The C++ engine does the inverse: it reads the command, mutates the linked-list and BST, then writes the new state. Algorithm modules (`greedy.py`, `backtracking.py`) are called directly from the UI as pure functions with no I/O.

TABLE I
LEVEL CONFIGURATION SUMMARY

Lvl	Grid	Enemies	ms/spawn	Budget	Heavy	Spawns
1	8×8	5	4000	4	No	1
2	8×8	6	3500	4	No	1
3	10×10	7	3500	5	Yes	1
4	10×10	8	3000	5	Yes	1
5	10×10	10	3000	6	Yes	2

interval. Greedy decisions are visualised in real time by filling the recommended cell in teal; enemy paths are drawn as blue outlines over the grid cells they traverse; yellow lines are rendered each step from each attacking tower to its nearest target, providing immediate feedback on which towers are active. The HUD panel on the right of the window lists towers in the order produced by the BST in-order traversal, making the data structure’s sorting property observable during play. Five levels of escalating difficulty are defined in `levels.py`, as summarised in Table I. Levels 1 and 2 use an 8×8 grid; levels 3–5 expand to 10×10 . Level 5 introduces a second spawn point at cell (0, 9), requiring two independent Backtracking calls per placement or step event. Two tower types are available—*base* (attack power 15, radius 2) and *heavy* (attack power 40, radius 3), the latter unlocked from level 3—and two enemy types—*normal* (HP 70, speed 1) and *fast* (HP 45, speed 2), the latter introduced in level 4.

III. RESULTS AND DISCUSSION

A. Testing Strategy and Unit Tests

The testing strategy followed an inside-out philosophy: each module was exercised in isolation before any integration testing was attempted. Unit tests were written informally as direct Python or C++ calls with manually constructed inputs whose expected outputs could be verified by inspection. Eleven test cases were executed and all passed; Table II summarises their scope, expected output, and outcome. For the Greedy module, the key assertions were that the scoring formula (1) produced values strictly decreasing with enemy distance and that the castle-proximity term correctly dominated on an empty board. For the Backtracking module, the critical property verified was that the returned path length equalled the Manhattan distance from spawn to castle on an obstacle-free grid, confirming that the pruning condition does not incorrectly discard the optimal solution. The linked-list tests

TABLE II
UNIT TEST SUMMARY

Component	Test case	Result
Greedy	No enemies: returns default cell	Pass
Greedy	Castle proximity dominates on empty board	Pass
Backtrack	Open grid: path len = Manhattan distance	Pass
Backtrack	Fully blocked grid: returns empty list	Pass
Backtrack	Pruning does not discard optimal path	Pass
Linked list	Push/pop maintains head, tail, size	Pass
Linked list	Remove-by-ID at boundary positions	Pass
Linked list	Advance increments position by speed	Pass
BST	In-order traversal yields ascending order	Pass
Engine	Step with no towers: enemy advances, no events	Pass
Engine	Tower kills enemy: <code>enemy_killed</code> emitted	Pass

verified that `ll_push_back` maintained the size counter, that `ll_remove_id` correctly updated head/tail pointers for boundary removals (first node, last node, single-node list), and that `ll_advance` incremented each enemy’s path index by its speed without overrunning the path array. The BST tests confirmed in-order traversal by inserting towers in random attack-power order and verifying that the output vector was sorted ascending.

B. Integration and Acceptance Tests

Integration testing verified the full JSON round-trip: Python wrote `input.json`, the engine binary was launched via `subprocess.run`, and the resulting `state.json` was read and compared against expected values. Three scenarios were covered: an enemy advancing one cell per step with no towers present; a tower dealing damage until the enemy’s HP reached zero and the `enemy_killed` event was emitted; and an enemy reaching the castle cell and the `castle_damaged` event being emitted with the correct HP decrement.

Acceptance testing consisted of playing all five levels to completion under two conditions: following the Greedy recommendation for every tower placement (normal play), and

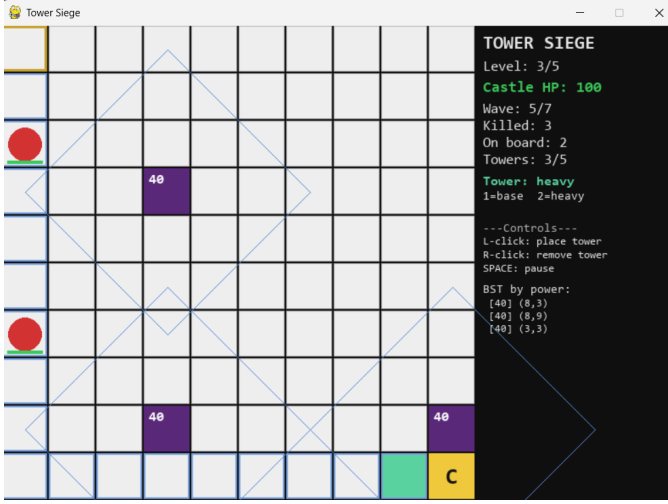


Fig. 2. Level-3 gameplay. The teal cell is the Greedy-recommended placement; the blue outline traces the Backtracking path from spawn (top-left) to castle (bottom-right). Every figure in this paper is referenced in the text.

placing towers arbitrarily without following the hint (adversarial play). Both sessions terminated correctly with either a win or a game-over screen and no unhandled Python exceptions. Fig. 2 shows a level-3 session with the greedy-recommended cell highlighted, the backtracking path drawn, and a heavy tower’s attack range visualised as a diamond outline. Fig. 3 shows the HUD panel with the BST in-order tower listing, confirming that towers are displayed in ascending attack-power order regardless of placement order.

C. The Greedy–Backtracking Conflict

The most significant finding of this project is an emergent interaction between the two algorithm modules that causes the game to freeze under normal play conditions.

Root cause. Examining equation (1), the castle-proximity term $\max(0, 8 - d_{\text{castle}}) \cdot 1.5$ assigns a score of up to 10.5 points to any cell one step from the castle, independently of enemy positions. On a board with few enemies, or when enemies are concentrated far from the castle, this term dominates the threat sum and the Greedy function consistently recommends cells in the bottom-right quadrant adjacent to the castle cell. The weight 1.5 was chosen intuitively to bias placements toward the defended objective, but it overrides the enemy-proximity signal in nearly all early-game and mid-game states.

Amplification by the placement guard. As the player follows the recommendations and towers fill the bottom-right area, the Backtracking pathfinder must route enemies through the remaining free cells, which form an increasingly narrow and convoluted corridor. Every attempted tower placement—including valid ones—triggers `placing_blocks_path`, which calls `find_path` for every spawn point. On a nearly full 10×10 grid the DFS may explore $O(4^L)$ partial paths before finding the first solution, where L is the length of the only available corridor. Because Python is single-threaded and the game loop runs on the same

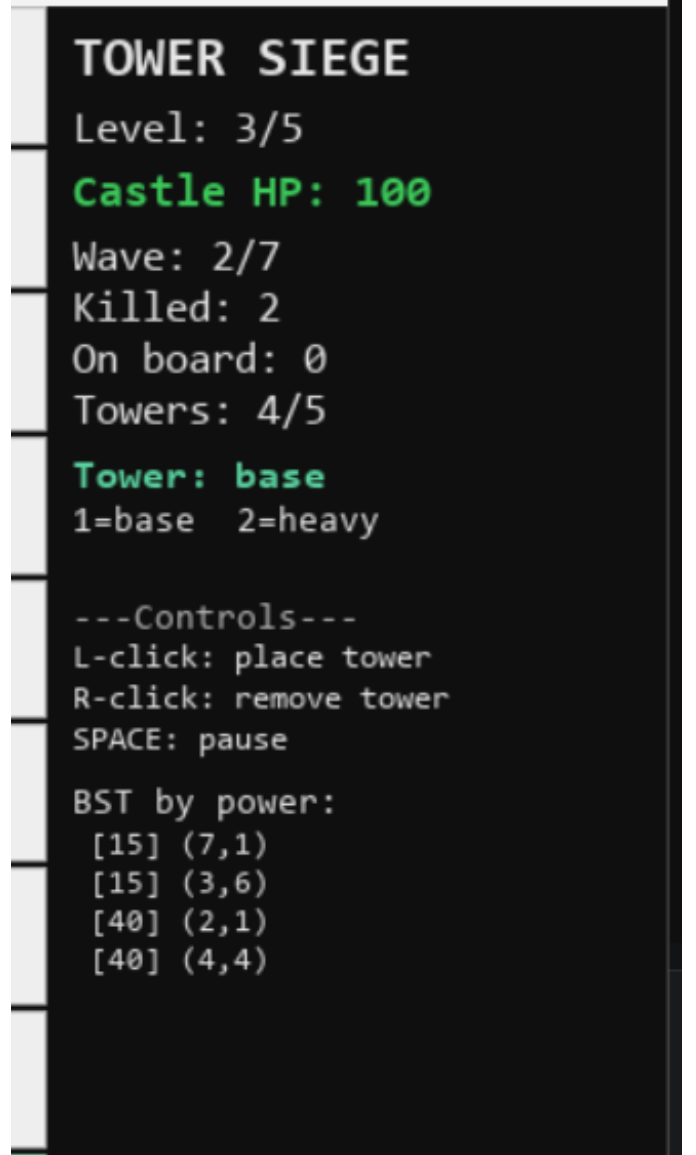


Fig. 3. HUD panel showing the in-order BST listing. Towers are displayed in ascending `attack_power` order regardless of the order they were placed, demonstrating the BST sorting property at runtime.

thread as the DFS, the Pygame event loop stops processing events for the duration of each `find_path` call. In the worst case observed during acceptance testing, this pause lasted several seconds on a level-5 grid with both spawn points active, making the game unresponsive and constituting a functional freeze.

Proposed remediations. Two targeted fixes are identified. First, re-weighting the castle-proximity term from 1.5 to 0.3–0.5 would allow the enemy-threat sum to dominate in most game states, directing the player toward cells along the middle of enemy routes rather than the castle perimeter. A more principled approach would replace the castle-distance proxy with a path-frequency influence map: the map is precomputed by flooding the grid from each spawn point and counting

how many routes pass through each cell, turning the Greedy into a genuine threat-aware heuristic. Second, replacing the Backtracking DFS with A^* —using the Manhattan distance to the goal as the admissible heuristic—would reduce the worst-case search effort from $O(4^L)$ to $O(b^d)$ with $b \approx 2.5$ on average for planar grid graphs; both `find_path` and `placing_blocks_path` would benefit immediately without any change to the JSON contract or the C++ engine.

IV. CONCLUSIONS

Tower Siege successfully demonstrates the integration of hand-rolled C++ data structures and Python algorithmic strategies in a fully playable, five-level tower-defense game. The linked-list enemy queue and BST tower registry both operated correctly and their runtime properties— $O(1)$ append, $O(n)$ removal, and $O(n)$ in-order traversal—were verified through unit and integration tests. Explicit raw-pointer management with the `init/free` idiom proved sufficient for correct heap handling throughout all test sessions, with no memory leaks detected during normal play. The JSON-mediated IPC contract enabled independent development and testing of the C++ and Python layers, a design decision confirmed to reduce debugging complexity during integration.

The primary limitation of the current implementation is the Greedy–Backtracking conflict documented in Section III-C. This interaction is not merely a performance issue but a design-level coupling between two components that were conceived independently: the Greedy heuristic shapes the tower layout in a way that systematically increases the cost of every subsequent Backtracking call, and no runtime mechanism alerts the player or the system to this degradation before it becomes a freeze. The interaction illustrates a general pitfall in algorithm composition: locally optimal decisions—placing towers near the castle to maximise the greedy score—can produce globally pathological states when the problem domains of two algorithms overlap.

Future work should address three open items. The Greedy scoring function should be revised to use a path-frequency influence map rather than raw castle-distance proximity, eliminating the root cause of the bias. The Backtracking pathfinder should be replaced by A^* with the Manhattan heuristic, converting the worst-case search cost from exponential to near-linear and making the placement-guard check fast regardless of grid occupancy. Finally, the BST should be extended with AVL rotations to guarantee $O(\log n)$ height for arbitrary attack-power ranges, preparing the engine for future variants where tower upgrades introduce a continuous power spectrum.

REFERENCES

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: The MIT Press, 2022.
- [2] P. E. Hart, N. J. Nilsson, and B. Raphael, “A formal basis for the heuristic determination of minimum cost paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [3] Python Software Foundation, “json — JSON encoder and decoder,” <https://docs.python.org/3/library/json.html>, 2024, accessed: 2026-06-03.
- [4] N. Lohmann, “JSON for modern C++,” <https://github.com/nlohmann/json>, 2024, accessed: 2026-06-03.

- [5] Pygame Community, “Pygame documentation,” <https://www.pygame.org/docs/>, 2024, accessed: 2026-06-03.